A decorative graphic on the left side of the slide, consisting of a black crosshair with a blue square in the top-left quadrant, a red square in the bottom-left quadrant, and a yellow square in the bottom-right quadrant.

Computer Programming II (CSE2035)

Lab #10: Linked List

Prof. Jaeseung Choi
Dept. of Computer Science and Engineering
Sogang University

Homework 10: Linked List (20pt)

- We will *reuse & extend* the Node and List class definitions from our lecture slide
 - Continued from our lecture
 - Check Node.hpp and List.hpp header files for the class definition

```
class Node {  
public:  
    int data;  
    Node *next;  
    Node(int i): data(i) {  
        next = NULL;  
    }  
};
```

```
class List {  
private:  
    Node *head; // The first node  
public:  
    ...  
};
```

Homework 10: Linked List (20pt)

- Here is the list of methods in our List class

```
class List {
private:
    Node *head; // The first node
public:
    List();
    ~List();
    void Print();
    // TODO from here!
    void InsertFront(int i);
    void InsertBack(int i);
    void InsertAt(int pos, int i);
    void RemoveFirst();
    void RemoveLast();
    void RemoveNth(int idx);
    bool IsEmpty();
    void Clear();
};
```

Provided in list.hpp

Insert() Methods

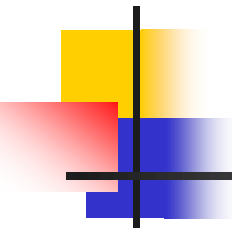
- Detailed specifications are given in `list.hpp`
- `InsertFront()` and `InsertBack()` were given in the lecture
 - But don't just copy-paste it; try to write the code on your own (for practice)
- **`InsertAt()` must insert a new node at the specified position**
 - `pos=0` means the front and `pos=N` means the back (where N is the number of nodes in the current list)

```
// Insert a new node at the front.
void InsertFront(int i);
// Insert a new node at the back.
void InsertBack(int i);
// Insert a new node at the specified position.
void InsertAt(int pos, int i);
```

Remove() Methods

- Detailed specifications are given in `list.hpp`
- `RemoveFirst()` and `RemoveLast()` were given in the lecture
 - But don't just copy-paste it; try to write the code on your own (for practice)
- `RemoveNth()` must remove the node at the specified index
 - `idx=0` means the first node and `idx=(N-1)` means the last node (where N is the number of nodes in the current list)

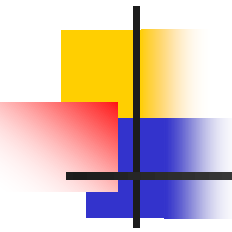
```
// Remove the first node.
void RemoveFirst();
// Remove the last node.
void RemoveLast();
// Remove the node at index 'idx'.
void RemoveNth(int idx);
```



Other Methods

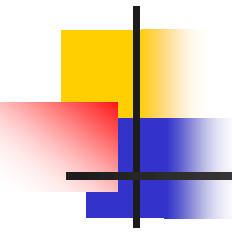
- Detailed specifications are given in `list.hpp`
- We must be able to (1) check whether the current list is empty, or (2) make the current list into an empty list

```
// Returns whether this list is empty.  
bool IsEmpty();  
// Make this list into an empty list.  
void Clear();
```



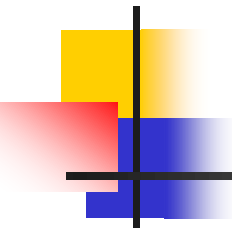
Note

- **Be careful for the corner cases and invalid inputs**
 - Ex) When the list is empty or has only one node
 - Ex) When the provided "pos" or "idx" arguments is invalid
- **Also, make sure that no memory leak occurs**
 - After `test()` in `main.cpp` returns, there shouldn't be any heap memory that is not deallocated yet
 - This time, I will grade your answer as wrong if a memory leak occurs
- **It's a good chance to practice your programming skills**

A decorative graphic consisting of overlapping yellow, red, and blue squares with a black crosshair.

Submission Guideline

- **Submit the completed `list.cpp` file to “Lab #10 Assignment” in cyber campus**
 - You can fix `main.cpp` to test your code, but don't submit it
 - Do not zip the directory, just upload this file directly
 - **Please do not change the file name**
 - If the submitted file doesn't compile with the "make" command, cannot give you any point
- **Deadline is 12/9 (Fri) 23:59**
- **Late submission is allowed until 12/12 (Mon) 23:59**
 - 20% deduction
- **Resubmission will be allowed until 12/16 (Fri) 23:59**
 - 40% deduction

A decorative graphic consisting of overlapping yellow, red, and blue squares with a black crosshair.

Q & A Session
